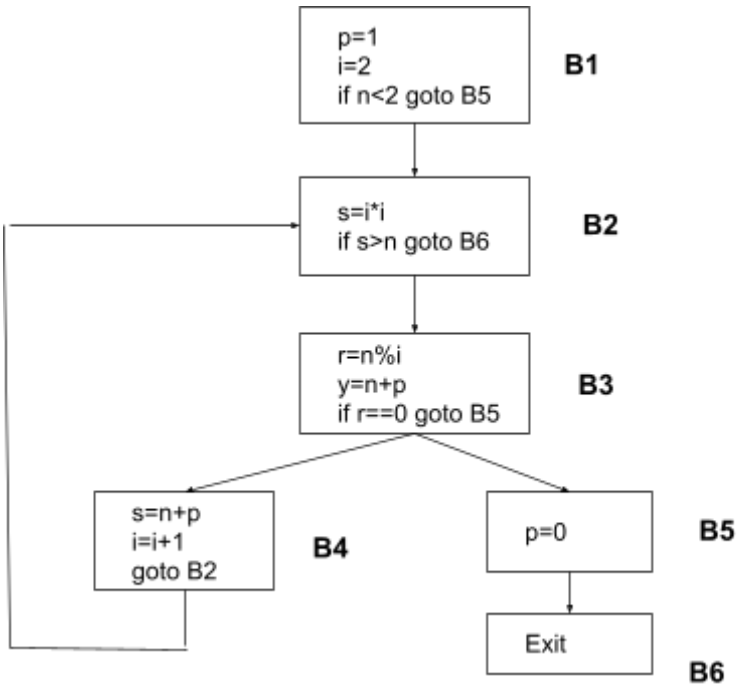
CS251: Introduction to Language Processing

Tierce-2 Solutions

Question-1: Total Marks: 25

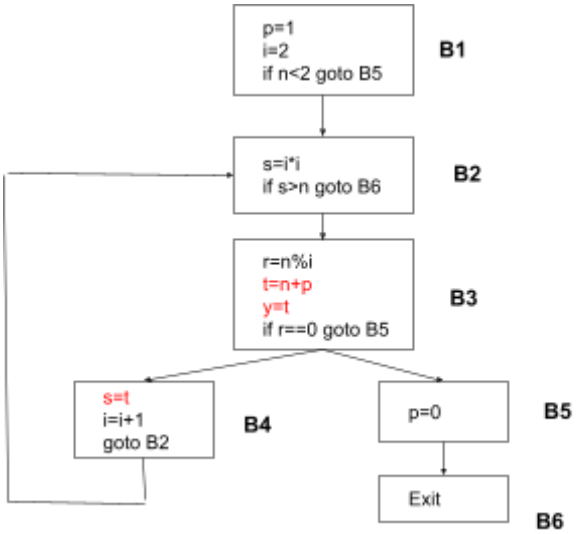
1) Marks: 12

Marking scheme:
Each basic block carries 1 Mark (Six blocks, so 6 Marks)
Each edge carries 1 Mark (Six edges, so 6 Marks).

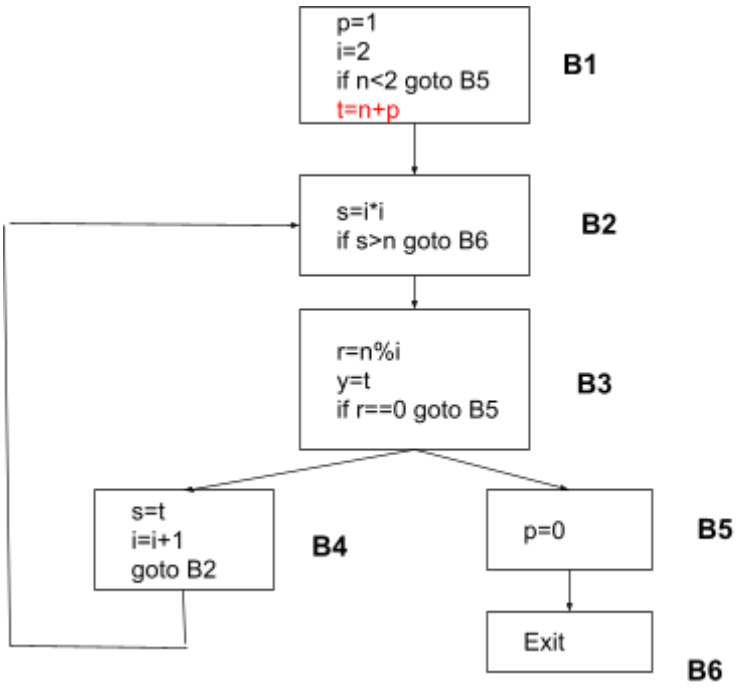


2) Marks: 7.5 Marks

Each instruction in the red colour carries 2.5 Marks.



3) The instruction t=n+p must be moved from B3 to B1 shown in the red colour. **This step carries 5.5 Marks.**



Question-2: Total Marks: 30

Part-(a): 4 Marks

Step-1 of the algorithm: 2.5 Marks
Step-2 of the algorithm: 1.5 Marks.

Labelling algorithm:

Since both the operands of the binary operation can be memory, we don't need explicit load instruction to copy the contents of memory to registers. As a result, the register requirement for leaf nodes is 0. Hence the modified algorithm is as follows.

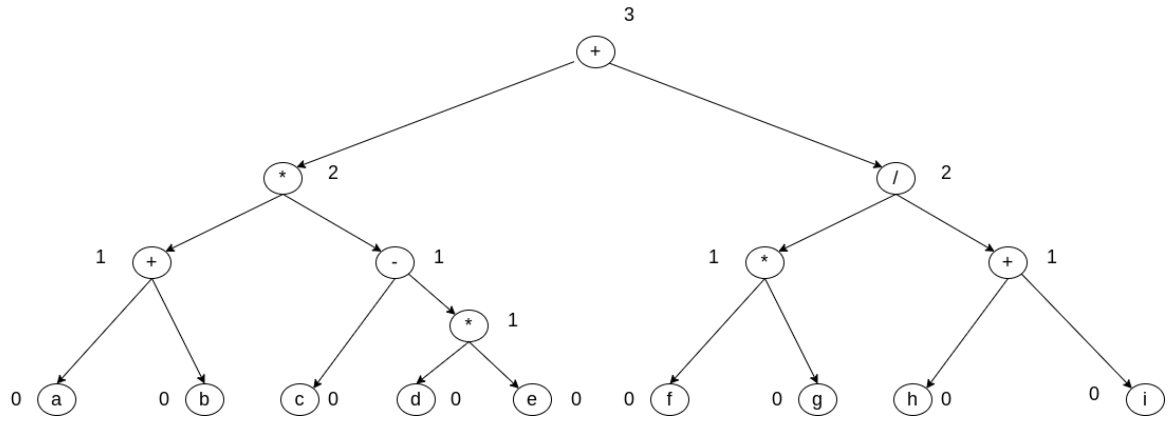
1. **Label each leaf node by 0**
2. If the labels of the children of a node n are l_1 and l_2 then

$$\begin{aligned} \text{label}(l) &= \max(l_1, l_2), && \text{if } l_1 \neq l_2 \\ &= l_1+1 && \text{Otherwise} \end{aligned}$$

Part-(b): 8 Marks

Marking Scheme: Each node and its corresponding label carries 0.5 Marks. Since there are 16 nodes, we assign 8 marks.

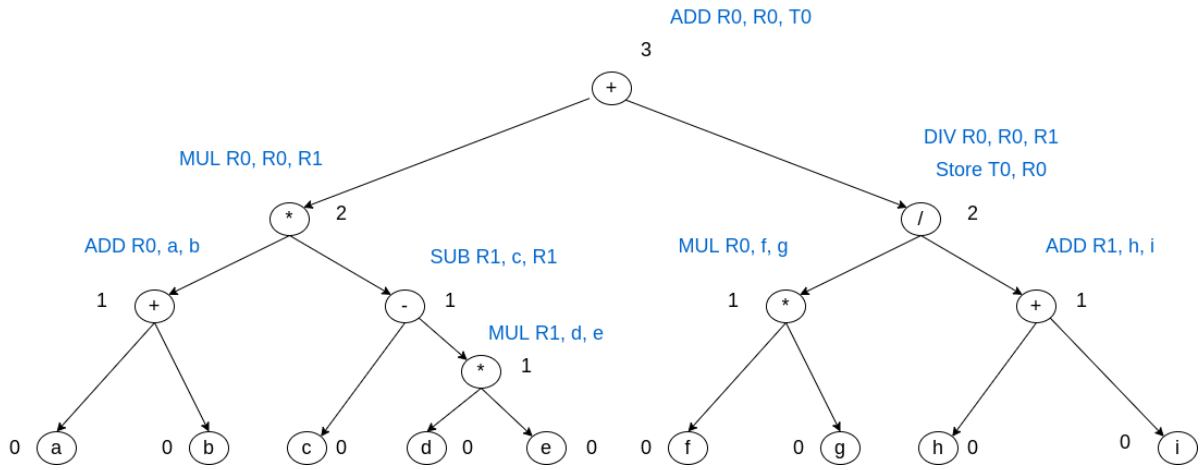
The labelled expression tree is shown below.



The minimum number of registers required to compute the expression ((a+b)*(c-d*e))+((f*g)/(h+i)) is **3**.

Part-(c): 18 Marks

Each generated assembly instruction carries 2 Marks. Please note that the order should be correct. Since there are 9 instructions, we assign 18 marks.



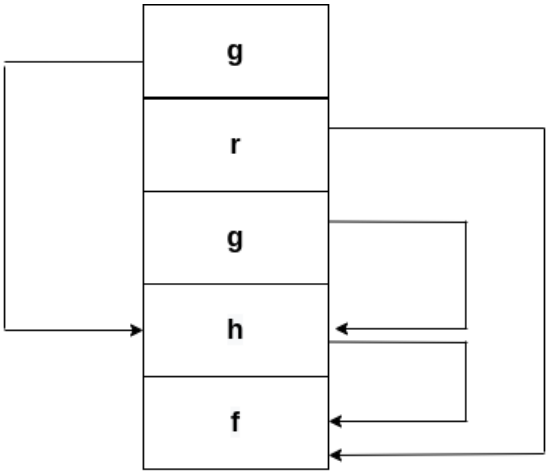
Generated assembly code.

```
MUL R0, f,g
ADD R1, h, i
DIV R0, R0, R1
Store T0, Ro
ADD R0, a, b
MUL R1, d, e
SUB R1, c, R1
MUL R0, R0, R1
ADD R0, R0, T0
```

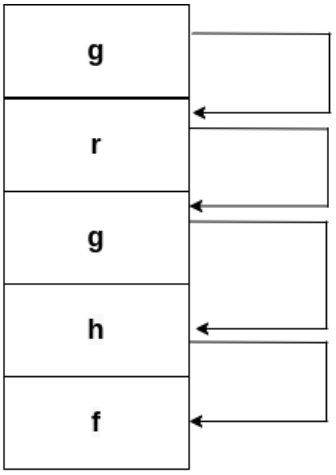
Question-3: Total Marks: 20

Part-(a): 12 Marks

Each part carries 6 marks. Each link in both parts carries 1.5 Marks. Please note the calling sequence must be correct.



(1) Static Scoping



(2) Dynamic Scoping

Part-(b): 8 Marks

Each sub part carries 4 marks (Correct answer: 1.5 + Correct justification: 2.5 Marks). Since there are sub parts, we assign 8 marks.

(1) Static Scoping:

The output is **25**.

Justification: In static scoping, the declaration of an identifier (v) is determined by the innermost textual declaration that is surrounded by the occurrence of v.

Since r() does not declare X, the declaration of X in r() is determined by its innermost nesting structure f(). Function f(), which has the declaration for X, and its latest value is 25. As a result, it prints 25.

(2) Dynamic Scoping:

The output is **75**.

Justification: In dynamic scoping, the declaration of an identifier (v) is determined by the most recent declaration that is seen during the execution leading the occurrence of v

Since r() does not declare X, the declaration of X in r() is determined by its previous calling function g(). Function g(), does not have a declaration of X, so it is determined its previous calling function h. Function h has the declaration for X, and its latest value is 75. As a result, it prints 75.

<u>Question-4:</u> Total Marks: 25

```
100:   i=0                    // 0.5 Marks
101:   t1=x+y                 // 0.5 Marks
102:   if i < t1 goto 104     // 0.5 Marks
103:   goto 126               // 0.5 Marks
104:   t2=i*x                 // 0.5 Marks
105:   t3=t2+y                // 0.5 Marks
106:   z=t3                   // 0.5 Marks
107:   if i>z goto 109        // 0.5 Marks
108:   goto 115               // 0.5 Marks
109:   t4=i*4                 // 1 Mark
110:   t5=a[t4]               // 1 Mark
111:   t6=z+t5                // 0.5 Marks
112:   t7=z*x                 // 0.5 Marks
113:   t8=y+z                 // 0.5 Marks
114:   t9=z+z                 // 0.5 Marks
115:   param t7               // 2 Marks
116:   param t8               // 2 Marks
117:   param t9               // 2 Marks
118:   t10=call function, 3   // 2 Marks
119:   w=t10                  // 0.5 Marks
120:   t11=w*z                // 0.5 Marks
121:   t12=z+t11              // 0.5 Marks
122:   z=t12                  // 0.5 Marks
123:   t13=i+1                // 2 Marks
124:   i=t13                  // 2 Marks
125:   goto 102               // 2 Marks
126:                          // 0.5 Marks
```